DO180 OpenShift

Complete Command Reference & Study Guide

Open Book Exam Reference — Search with Ctrl+F

CHAPTERS COVERED:

CHAPTER 1: Key Concepts — Kubernetes & OpenShift

What is Kubernetes?

Kubernetes is a platform that runs and manages containerized applications across multiple servers (nodes). Instead of telling it HOW to do things, you tell it WHAT you want (declarative), and it figures out how to get there and keeps it that way.

Key Terms to Know

Command	What it does
`Container`	A running process isolated from everything else. Created from a container image.
`Pod`	The smallest unit in Kubernetes. One or more containers that share network/storage.
`Node`	A server (physical or virtual) in the cluster. Runs your pods.
`Control Plane Node`	The 'brain' — manages scheduling and cluster state.
`Compute Node`	Where your apps actually run.
`Namespace / Project`	A logical group to isolate resources. OpenShift adds 'Projects' on top of namespaces.
`Deployment`	Tells Kubernetes to run N copies of a pod and keep them running.
`ReplicaSet`	Ensures a certain number of pod copies are always running (managed by Deployment).
`Service`	A stable IP address that points to a group of pods. Load balances traffic.
`Route`	Exposes a service to the internet with a hostname (OpenShift-specific).
`Operator`	A packaged app that manages other apps in Kubernetes automatically.
`etcd`	The database that stores the entire cluster state — Kubernetes' memory.
`CRI-O`	The container runtime that actually starts/stops containers in OpenShift.

Kubernetes vs OpenShift

OpenShift = Kubernetes + extra features. Everything in Kubernetes works in OpenShift, but OpenShift adds: built-in registry, CI/CD pipelines, Routes, Projects, better security (SCCs), and a web console.

Replicas — Simple Explanation

Replicas = copies of the same pod running at the same time. If you set replicas: 3, Kubernetes runs 3 identical pods. This gives you high availability (if one crashes, others keep running) and load balancing (traffic spread across all copies).

OpenShift Editions (Self-Managed)

Command	What it does
`Kubernetes Engine`	Basic tier — core Kubernetes + security hardening
`Container Platform`	Middle tier — adds developer console, monitoring, pipelines (most common)
`Platform Plus`	Top tier — adds advanced cluster management, security, private registry

Managed OpenShift (Cloud)

Command	What it does
`ROSA`	Red Hat OpenShift on AWS
`ARO`	Azure Red Hat OpenShift
`OpenShift on IBM Cloud`	IBM's managed version
`OpenShift Dedicated`	Single-tenant, managed by Red Hat on AWS or GCP

CHAPTER 2: CLI Commands — oc and kubectl

Login & Project Commands

⚠️ **NOTE:** Always log in first before running any oc command. Use oc (not kubectl) for OpenShift-specific features.

Command	What it does
`oc login -u USER -p PASS https://api.ocp4.example.com:6443`	Log in to the cluster with username and password
`oc login --token=TOKEN --server=URL`	Log in with an OAuth token (more secure)
`oc login --web`	Log in via browser
`oc logout`	Log out of the cluster
`oc whoami`	Show currently logged-in user
`oc whoami --show-console`	Show the URL of the web console
`oc project PROJECT_NAME`	Switch to a different project
`oc new-project PROJECT_NAME`	Create a new project and switch to it
`oc projects`	List all projects you have access to
`oc status`	Overview of current project — shows services, deployments, issues
`oc version`	Show client and server version
`oc cluster-info`	Show the URL of the control plane

Get / Describe / Explain

💡 **TIP:** oc get shows a summary. oc describe shows full details including Events. oc explain shows documentation for a resource type.

Command	What it does
`oc get pods`	List all pods in current project
`oc get pods -o wide`	List pods with extra info (IP, node)
`oc get pods -o yaml`	Show full pod definition in YAML format
`oc get pods -o json`	Show full pod definition in JSON format
`oc get pods -l app=myapp`	Filter pods by label
`oc get pods --show-labels`	Show labels on each pod
`oc get pods -n NAMESPACE`	List pods in a different namespace
`oc get pods -A`	List pods in ALL namespaces

`oc get all`	Show most important resources in current project
`oc get deployment`	List deployments
`oc get services`	List services (svc)
`oc get routes`	List routes
`oc get nodes`	List cluster nodes
`oc get pvc`	List persistent volume claims
`oc get pv`	List persistent volumes
`oc get secrets`	List secrets
`oc get configmaps`	List config maps (cm)
`oc get events`	Show events in current project
`oc get events --sort-by .metadata.creationTimestamp`	Show events sorted by time
`oc get clusteroperators`	List all built-in cluster operators (needs admin)
`oc get operators`	List add-on operators installed by users
`oc describe pod POD_NAME`	Full details about a pod including Events
`oc describe deployment DEPLOY_NAME`	Full details about a deployment
`oc describe node NODE_NAME`	Full details about a node
`oc describe project PROJECT_NAME`	Show project details including UID/GID ranges
`oc explain pod`	Show documentation for the pod resource type
`oc explain pod.spec`	Show documentation for pod spec fields
`oc explain pods.spec.containers.resources`	Show docs for a specific nested field
`oc api-resources`	List all resource types the cluster supports
`oc api-resources --namespaced`	List only namespaced resources
`oc api-resources --api-group apps`	List resources from the apps API group
`oc api-versions`	List all supported API versions

Filtering Output with jq and jsonpath

💡 **TIP:** Use -o json | jq to extract specific fields. Use -o jsonpath for quick single-field extraction without installing jq.

```
oc get pods -o json | jq '.items[0].status.podIP'
```

→ Get the IP of the first pod

```
oc get pods -o jsonpath='{.items[0].status.podIP}{"\n"}'
```

→ Same thing using jsonpath (no jq needed)

```
oc get node master01 -o jsonpath='{.status.allocatable.pods}{"\n"}'
```

→ How many pods can fit on master01

```
oc get node master01 -o json | jq '.status.conditions'
```

→ Check node health conditions (MemoryPressure, DiskPressure, Ready, etc.)

```
oc get deployment myapp -o
jsonpath='{.spec.template.spec.containers[0].resources.requests.memory}{"\n"}'
```

→ Check memory request of a deployment

Node Health Conditions — What They Mean

Command	What it does
`Ready = True`	Node is healthy and accepting pods ✅
`MemoryPressure = True`	Node is LOW on memory — problem ❌
`DiskPressure = True`	Node is LOW on disk space — problem ❌
`PIDPressure = True`	Too many processes running — problem ❌
`NetworkUnavailable = True`	Network not configured correctly — problem ❌

💡 **TIP:** Check node conditions: oc get node master01 -o json | jq '.status.conditions'

Check Valid Control Plane Node Condition Types

```
kubectl get node master01 -o jsonpath='{.status.conditions[*].type}'
```

Output: MemoryPressure DiskPressure PIDPressure NetworkUnavailable Ready

Cluster Health Commands

Command	What it does
`oc adm top pods -A --sum`	Show CPU and memory usage of all pods
`oc adm top node`	Show CPU and memory usage of nodes
`oc adm top pods POD -n NS --containers`	Show resource usage per container in a pod
`oc adm node-logs master01 -u crio --tail 1`	Show last 1 log entry from crio service on master01
`oc adm node-logs master01 -u kubelet --tail 5`	Show last 5 kubelet log entries
`oc adm must-gather --dest-dir /home/student/must-gather`	Collect cluster debug info for support
`oc adm inspect clusteroperator/openshift-apiserver --since 10m`	Collect debug info for specific operator

CHAPTER 3: Containers, Pods, and Container Images

Running Pods

Command	What it does
`oc run POD_NAME --image IMAGE`	Create a pod from an image
`oc run POD_NAME --image IMAGE -it -- /bin/bash`	Create pod and open interactive shell
`oc run POD_NAME --image IMAGE --restart Never -- date`	Run pod once, execute 'date' command
`oc run POD_NAME --image IMAGE --rm --restart Never -- date`	Run pod, execute command, then DELETE the pod
`oc run POD_NAME --image IMAGE --env VAR=value`	Create pod with environment variable
`oc run POD_NAME --image IMAGE --command -- cmd arg1`	Override the default command in the image
`oc run POD_NAME --image IMAGE --dry-run=client -o yaml`	Generate YAML without actually creating the pod

Executing Commands in Running Pods

Command	What it does
`oc exec POD_NAME -- date`	Run 'date' command inside the pod
`oc exec POD_NAME -it -- /bin/bash`	Open interactive shell in the pod
`oc exec POD_NAME -c CONTAINER_NAME -- cmd`	Run command in a specific container (multi-container pods)
`oc exec POD_NAME -- ls /var/www/html`	List files in a directory inside the pod
`oc exec POD_NAME -- cat /etc/file.conf`	Read a file inside the pod
`oc exec POD_NAME -- head /home/database-files/insertdata.sql`	Show first 10 lines of a file inside pod
`oc exec -it POD_NAME -- /bin/bash -c 'whoami && id'`	Check user identity inside the pod
`oc rsh POD_NAME`	Open a remote shell in the pod (simpler than exec)
`oc attach POD_NAME -it`	Attach to a running interactive session in the pod

Copying Files To/From Pods

Command	What it does

`oc cp /local/file.txt POD_NAME:/remote/path/`	Copy file FROM your machine TO the pod
`oc cp POD_NAME:/remote/file.txt /local/path/`	Copy file FROM the pod TO your machine
`oc rsync POD_NAME:/var/www/ /tmp/web_files`	Sync directory from pod to local

⚠️ **NOTE:** The tar binary must exist inside the container for oc cp to work.

Pod Logs

Command	What it does
`oc logs POD_NAME`	Show logs for a pod
`oc logs POD_NAME --tail=10`	Show last 10 lines of logs
`oc logs POD_NAME -f`	Follow/stream logs in real time
`oc logs POD_NAME -c CONTAINER_NAME`	Logs from specific container in multi-container pod
`oc logs POD_NAME -p`	Show logs from the PREVIOUS (crashed) container instance
`oc logs deploy/DEPLOY_NAME`	Show logs from the deployment's current pod

Debugging Pods

Command	What it does
`oc debug pod/POD_NAME`	Create a debug copy of the pod with a shell
`oc debug deployment/DEPLOY_NAME`	Debug pod from a deployment
`oc debug node/master01`	Debug a node (admin only) — then run chroot /host
`oc debug pod/POD_NAME --as-user=1000000`	Debug running as a specific UID
`oc debug pod/POD_NAME --as-root`	Debug running as root

💡 **TIP:** After 'oc debug node/master01', always run 'chroot /host' to access host binaries like crictl and systemctl.

Port Forwarding

```
oc port-forward POD_NAME 8080:8080
```

Forwards local port 8080 to pod port 8080. Useful for testing without creating a route. Press Ctrl+C to stop.

Deleting Pods and Resources

Command	What it does
`oc delete pod POD_NAME`	Delete a pod (if managed by deployment, it recreates automatically)
`oc delete pod POD_NAME --now`	Delete immediately (grace period = 1 second)
`oc delete pod POD_NAME --force`	Force delete without waiting for confirmation
`oc delete pod -l app=myapp`	Delete all pods with label app=myapp
`oc delete pods --all`	Delete ALL pods in current project
`oc delete project PROJECT_NAME`	Delete entire project and all its resources
`oc delete all -l team=red`	Delete all resources with label team=red

Skopeo — Inspect Container Images Without Running Them

💡 **TIP:** Always run 'skopeo login REGISTRY' before using skopeo commands on private registries.

Command	What it does
`skopeo login registry.ocp4.example.com:8443`	Log in to a container registry
`skopeo list-tags docker://REGISTRY/IMAGE`	List all available tags for an image
`skopeo inspect docker://REGISTRY/IMAGE:TAG`	Show full image metadata (JSON output)
`skopeo inspect docker://REGISTRY/IMAGE:TAG | jq '.Digest'`	Get the image hash/SHA256
`skopeo inspect docker://REGISTRY/IMAGE:TAG | jq '.Env'`	Get environment variables in the image
`skopeo inspect docker://REGISTRY/IMAGE:TAG | jq 'keys'`	List all available fields in the output
`skopeo inspect --config docker://IMAGE | jq '.config.User'`	Get the USER (UID) defined in image
`skopeo inspect --config docker://IMAGE | jq '.config.ExposedPorts'`	Get exposed ports
`skopeo copy docker://SOURCE docker://DESTINATION`	Copy image from one registry to another
`skopeo delete docker://REGISTRY/IMAGE:TAG`	Delete an image from a registry

oc image — Inspect Images via OpenShift

Command	What it does

`oc image info IMAGE`	Show image metadata including User, Command, Ports, Env
`oc image info IMAGE | grep User`	Check what UID the image runs as
`oc image info IMAGE | grep -i command`	Check what command the image runs
`oc image info IMAGE | grep 'Exposes Ports'`	Check exposed ports
`oc image info IMAGE --filter-by-os amd64`	Specify architecture for multi-arch images

KEY CONCEPT: UID Rules — Who Runs as What?

Scenario	Regular User Deploys	Admin Deploys
Image USER = 1001	IGNORED — random high UID from namespace range	HONOURED — runs as UID 1001
Image USER = 0 (root)	IGNORED — still gets random UID (non-root)	HONOURED — runs as root (UID 0) = PRIVILEGED
No USER defined	Random UID from namespace range	Runs as ROOT — PRIVILEGED ⚠️
Image USER = 26 or 27 (system UID)	Random UID from namespace	Runs as system account (postgres=26, mysql=27) = PRIVILEGED

💡 **TIP:** Check UID: oc image info IMAGE | grep User | No output or UID 0 = privileged when admin deploys

CHAPTER 4: Deploy and Network Applications

Creating Deployments

Command	What it does
`oc create deployment NAME --image IMAGE`	Create a deployment from an image
`oc create deployment NAME --image IMAGE --replicas 3`	Create deployment with 3 replicas
`oc create deployment NAME --image IMAGE --port 8080`	Create deployment exposing port 8080
`oc new-app --image IMAGE -l label=value`	Create deployment + service from image (OpenShift-specific)
`oc new-app --template TEMPLATE_NAME`	Deploy from a pre-built template
`oc new-app --file=./my-app.yaml`	Deploy from a YAML file
`oc create -f manifest.yaml`	Create any resource from a YAML file
`oc apply -f manifest.yaml`	Create or UPDATE resource from a YAML file

Managing Deployments

Command	What it does
`oc scale deployment/NAME --replicas 3`	Scale to 3 replicas
`oc set image deployment/NAME CONTAINER=NEW_IMAGE`	Update container image in deployment
`oc set env deployment/NAME VAR=value`	Set environment variable on deployment
`oc set env deployment/NAME --from secret/SECRET_NAME --prefix MYSQL_`	Set env vars from a secret
`oc set resources deployment/NAME --requests memory=512Mi`	Set memory request
`oc set resources deployment/NAME --limits memory=1Gi`	Set memory limit
`oc set probe deployment/NAME --readiness --get-url http://:3000/health`	Add readiness probe
`oc set probe deployment/NAME --liveness --get-url http://:3000/health`	Add liveness probe
`oc edit deployment/NAME`	Edit deployment YAML in your editor (vim by default)

`oc patch deployment NAME --type=json -p='[...]'`	Patch a specific field in a deployment
`oc rollout status deployment/NAME`	Watch rollout progress
`oc rollout history deployment/NAME`	See rollout history
`oc rollout undo deployment/NAME`	Roll back to previous version

💡 **TIP:** In vim (oc edit): save & exit = Esc then :wq + Enter | exit without saving = Esc then :q! + Enter

Jobs and CronJobs

Command	What it does
`oc create job JOB_NAME --image IMAGE -- /bin/bash -c 'date'`	Create a one-time job
`oc create cronjob NAME --image IMAGE --schedule '*/1 * * * *' -- date`	Create a recurring job (every minute)
`oc get jobs`	List jobs
`oc logs job/JOB_NAME`	Show logs from a job
`oc delete job JOB_NAME`	Delete a job (also deletes its pods)

Services — Internal Networking

A Service gives a stable IP to a group of pods. Pods can find each other by service name, not IP (which changes). Services load balance traffic across all matching pods.

Command	What it does
`oc expose deployment/NAME`	Create a ClusterIP service for a deployment
`oc expose deployment/NAME --name svc-name --port 8080 --target-port 8080`	Create service with specific ports
`oc get services`	List services
`oc get service NAME -o wide`	Show service with its selector label
`oc get endpoints`	Show which pod IPs each service points to
`oc delete service NAME`	Delete a service

DNS Names for Services

Every service gets a DNS name that pods can use. The formats:

- SVC-NAME — works within the same project

- SVC-NAME.PROJECT-NAME — works from any project
- SVC-NAME.PROJECT-NAME.svc.cluster.local — full FQDN, works everywhere

Routes — External Access

A Route exposes a service to the internet via a hostname. Think of it as the front door to your app.

Command	What it does
`oc expose service SVC_NAME`	Create a route for a service (auto-generates hostname)
`oc expose service SVC_NAME --hostname myapp.apps.ocp4.example.com`	Create route with specific hostname
`oc get routes`	List routes and their hostnames
`oc delete route ROUTE_NAME`	Delete a route
`oc annotate route ROUTE_NAME router.openshift.io/cookie_name=myapp`	Enable sticky sessions on a route

Ingress Objects

Command	What it does
`oc create ingress NAME --rule='host/*=service:port'`	Create an Ingress object
`oc get ingress`	List ingress objects
`oc annotate ingress NAME ingress.kubernetes.io/affinity=cookie`	Enable sticky sessions on ingress
`oc delete ingress NAME`	Delete an ingress object

💡 **TIP:** Creating an Ingress automatically creates a managed Route. Deleting the Ingress deletes the Route too.

CHAPTER 5: Storage — ConfigMaps, Secrets, PVCs, StatefulSets

ConfigMaps — Store Non-Sensitive Config Data

ConfigMaps store configuration data (not secrets) that can be injected into pods as environment variables or files.

Command	What it does
`oc create configmap NAME --from-literal key=value`	Create configmap from key-value pair
`oc create configmap NAME --from-literal k1=v1 --from-literal k2=v2`	Create configmap with multiple keys
`oc create configmap NAME --from-file /path/to/file.conf`	Create configmap from a file
`oc create cm NAME --from-file /path/to/config.sql`	Short form: 'cm' instead of 'configmap'
`oc get configmaps`	List all configmaps
`oc describe configmap NAME`	Show configmap contents
`oc set volume deployment/NAME --add --type configmap --configmap-name CM_NAME --mount-path /path`	Mount configmap as files in pod
`oc extract configmap/NAME --to /tmp/mydir --confirm`	Extract configmap values to local files
`oc set data configmap/NAME --from-file /tmp/file`	Update configmap from a local file
`oc delete configmap NAME`	Delete a configmap

Secrets — Store Sensitive Data

Secrets store passwords, tokens, SSH keys. Values are Base64-encoded (NOT encrypted). Access is restricted.

⚠️ **NOTE:** Secrets are Base64-encoded, not encrypted. Anyone with access to the secret can decode it with: echo VALUE | base64 --decode

Command	What it does
`oc create secret generic NAME --from-literal user=myuser --from-literal pass=mypass`	Create a generic secret
`oc create secret generic NAME --from-file key=/path/to/file`	Create secret from a file

`oc create secret tls NAME --cert /path/cert --key /path/key`	Create a TLS secret
`oc get secrets`	List secrets
`oc describe secret NAME`	Show secret metadata (values are hidden)
`oc set volume deployment/NAME --add --type secret --secret-name NAME --mount-path /path`	Mount secret as files in pod
`oc set env deployment/NAME --from secret/NAME --prefix MYSQL_`	Set env vars from secret (adds prefix)
`oc extract secret/NAME --to /tmp/dir --confirm`	Extract secret values to local files
`oc delete secret NAME`	Delete a secret

Persistent Volume Claims (PVCs) — Persistent Storage

Pods lose data when they restart. PVCs give pods storage that survives restarts and pod deletions.

Command	What it does
`oc get storageclass`	List available storage classes
`oc describe storageclass CLASS_NAME`	Show details about a storage class
`oc get pvc`	List persistent volume claims
`oc get pv`	List persistent volumes (cluster-wide)
`oc describe pvc PVC_NAME`	Show PVC details including which pod uses it
`oc set volumes deployment/NAME --add --name vol-name --type pvc --claim-mode rwo --claim-size 1Gi --mount-path /var/data --claim-name PVC_NAME`	Add a new PVC to a deployment
`oc set volumes deployment/NAME --add --type pvc --mount-path /var/data --name vol-name --claim-name EXISTING_PVC`	Attach existing PVC to deployment
`oc create -f pvc.yaml`	Create PVC from a YAML file
`oc delete pvc PVC_NAME`	Delete a PVC

Access Modes

Command	What it does
`ReadWriteOnce (RWO)`	One node can read/write. Multiple pods on that node can share it.

`ReadWriteOncePod (RWOP)`	Only ONE pod in the entire cluster can read/write.
`ReadOnlyMany (ROX)`	Many nodes can read but NOT write.
`ReadWriteMany (RWX)`	Many nodes can read AND write. Good for shared storage (NFS).

Reclaim Policies

Command	What it does
`Delete`	When PVC is deleted, the PV and data are also deleted automatically.
`Retain`	When PVC is deleted, PV and data are kept. Admin must manually clean up.

StatefulSets — For Apps That Need Unique Storage

Unlike Deployments (where all pods share one PVC), StatefulSets give each pod its OWN dedicated PVC. Used for databases and other stateful apps.

Command	What it does
`oc create -f statefulset.yaml`	Create a stateful set from YAML (cannot create imperatively)
`oc get statefulset`	List stateful sets
`oc scale statefulset/NAME --replicas 4`	Scale a stateful set
`oc delete statefulset NAME`	Delete stateful set (PVCs are NOT deleted automatically)
`oc get pvc -l app=database`	List PVCs created by a stateful set

💡 **TIP:** StatefulSet pod names are predictable: dbserver-0, dbserver-1, dbserver-2. If pod-0 is deleted, a new pod-0 is created and mounts data-dbserver-0 PVC — data is preserved!

Headless Service for StatefulSets

StatefulSets need a headless service (clusterIP: None). This gives each pod a unique DNS name:

```
dbserver-0.mysql-svc.namespace.svc.cluster.local
```

This lets apps talk to specific pods by name (important for database replication).

CHAPTER 6: Reliability — Probes, Resource Limits, HPA

Resource Requests and Limits

Requests = minimum resources a pod needs (used for scheduling). Limits = maximum resources a pod can use.

⚠️ **NOTE:** If memory request is too high (e.g. 8Gi) and no node has that much free, the pod stays in Pending state forever.

Command	What it does
`oc set resources deployment/NAME --requests memory=512Mi`	Set memory request to 512 MiB
`oc set resources deployment/NAME --requests cpu=100m`	Set CPU request to 100 millicores
`oc set resources deployment/NAME --limits memory=1Gi --limits cpu=500m`	Set both memory and CPU limits
`oc get deployment NAME -o jsonpath='{.spec.template.spec.containers[0].resources.requests.memory}'`	Check current memory request
`oc describe pod POD_NAME`	See Events section to find scheduling failures

Health Probes

Probes tell Kubernetes when your app is ready and when it's healthy:

- Readiness Probe: Is the app ready to receive traffic? (Pod won't get traffic until this passes)
- Liveness Probe: Is the app still alive? (Pod is restarted if this fails)

Command	What it does
`oc set probe deployment/NAME --readiness --get-url http://:3000/health --initial-delay-seconds 7`	Add readiness probe on HTTP endpoint, wait 7s before first check
`oc set probe deployment/NAME --liveness --get-url http://:8080/health`	Add liveness probe
`oc set probe deployment/NAME --readiness --open-tcp=3306`	Add readiness probe on a TCP port
`oc set probe deployment/NAME --readiness -- cat /tmp/healthy`	Add readiness probe that runs a command
`oc set probe deployment/NAME --remove --readiness`	Remove readiness probe

💡 **TIP:** The readiness probe prevents 'app is still starting' errors when scaling up. Always set --initial-delay-seconds to the app's startup time.

Horizontal Pod Autoscaler (HPA)

HPA automatically scales your deployment up or down based on CPU or memory usage.

Command	What it does
`oc autoscale deployment/NAME --min 1 --max 5 --cpu-percent 60`	Auto-scale based on CPU (60% threshold)
`oc apply -f hpa.yaml`	Create HPA from a YAML file (for memory-based scaling)
`oc get hpa`	List HPAs
`oc get hpa NAME`	Show HPA with current metrics (TARGETS column)
`watch oc get hpa NAME`	Watch HPA metrics update in real time
`oc delete hpa NAME`	Delete an HPA

HPA YAML for Memory-Based Scaling

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: longload
spec:
  maxReplicas: 3
  minReplicas: 1
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: longload
  metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 60
```

CHAPTER 7: Application Updates — Image Streams & Triggers

Floating vs Fixed Image Tags

Command	What it does
`latest (floating tag)`	Always points to the most recent image — can change unexpectedly!
`1-222 (fixed tag)`	Always points to the exact same image build — stable and predictable
`1 (alias/floating)`	Can be reassigned to point to a different fixed tag like 1-234

💡 **TIP:** Use fixed tags (like 1-222) for production to prevent accidental updates when someone pushes a new image.

Image Stream Commands

Command	What it does
`oc get imagestream`	List image streams in current project
`oc describe imagestream NAME`	Show image stream details and tags
`oc create istag STREAM:TAG --from-image REGISTRY/IMAGE:TAG`	Create an image stream tag pointing to a registry image
`oc tag --alias STREAM:NEW_TAG STREAM:OLD_TAG`	Make NEW_TAG an alias pointing to OLD_TAG
`oc tag STREAM:TAG --scheduled`	Auto-import new versions periodically

Updating Deployment Images

Command	What it does
`oc set image deployment/NAME CONTAINER=REGISTRY/IMAGE:TAG`	Update the image used by a deployment
`oc get deployment/NAME -o wide`	Verify which image the deployment is using

Image Triggers

An image trigger makes a deployment automatically redeploy when an image stream tag changes.

Command	What it does

`oc set triggers deployment/NAME --from-image STREAM:TAG --containers CONTAINER`	Add image trigger to deployment
`oc set triggers deployment/NAME`	Show current triggers on a deployment
`oc set triggers deployment/NAME --remove --from-image STREAM:TAG`	Remove an image trigger

Full Image Update Workflow

1. Create new image stream tag pointing to the new image:

```
oc create istag php-ssl:1-234 --from-image
registry.ocp4.example.com:8443/redhattraining/php-ssl:1-234
```

2. Move the alias to point to the new tag:

```
oc tag --alias php-ssl:1-234 php-ssl:1
```

3. OpenShift detects the change and automatically redeploys app2 (because it has an image trigger on php-ssl:1)

4. Verify the rollout:

```
oc describe deployment/app2
```

Look at Events section — you should see 'Scaled up replica set...' for the new version

QUICK REFERENCE — Most Used Commands

Login & Setup

```
oc login -u developer -p developer https://api.ocp4.example.com:6443
oc project PROJECT_NAME
skopeo login registry.ocp4.example.com:8443   # Username: developer / Password: developer
```

Image Inspection Cheat Sheet

Command	What it does
`What command does image run?`	oc image info IMAGE → look for Command: line
`What UID does image use?`	oc image info IMAGE | grep User
`What ports does image expose?`	oc image info IMAGE → look for Exposes Ports:
`What env vars does image have?`	skopeo inspect docker://IMAGE | jq '.Env'
`What is the image SHA/hash?`	skopeo inspect docker://IMAGE | jq '.Digest'
`What tags are available?`	skopeo list-tags docker://REGISTRY/REPO
`Check all images for privileged UID`	for image in img1 img2; do echo $image; oc image info $image | grep User; done

Troubleshooting a Failing Pod

Step 1 — Check pod status:

```
oc get pods
```

Step 2 — Check logs:

```
oc logs POD_NAME
```

Step 3 — Check events:

```
oc get events
```

Step 4 — Get detailed description:

```
oc describe pod POD_NAME
```

Step 5 — Debug with a shell:

```
oc debug pod/POD_NAME
```

Common Error Causes

Command	What it does
`CrashLoopBackOff`	App is crashing — check logs with: oc logs POD_NAME
`ImagePullBackOff`	Can't pull the image — wrong tag? Wrong registry? Need to login?

`Pending (never starts)`	Not enough resources (CPU/memory) on node — check events
`Permission denied errors`	Container needs root but gets non-root UID — image needs fixing
`No such file or directory`	Wrong mount path — check where configmap/secret was mounted
`Connection refused`	Wrong port, service not ready, or app still starting

Key Flags to Remember

Command	What it does
`-n NAMESPACE`	Run command in a different namespace/project
`-A or --all-namespaces`	Run command across ALL namespaces
`-o wide`	Show extra columns in output
`-o yaml / -o json`	Show full resource definition
`-o jsonpath='{...}'`	Extract a specific field
`-l label=value`	Filter by label
`--show-labels`	Show labels on resources
`--tail=N`	Show last N lines of logs
`-f`	Follow/stream logs in real time
`-it`	Interactive terminal (needed for shells)
`--rm`	Delete pod after it exits
`--restart Never`	Don't restart pod when it finishes
`--dry-run=client -o yaml`	Generate YAML without creating resource

GLOSSARY — Key Terms in Simple English

Command	What it does
`etcd`	The database that stores the entire cluster state. Kubernetes' memory.
`CRI-O`	The container runtime that actually starts/stops containers in OpenShift.
`Grafana`	Dashboards that show metrics as visual graphs and charts.
`Prometheus`	Collects and stores metrics (CPU, memory, traffic) by scraping your apps.
`CoreOS (RHCOS)`	The minimal, container-optimized OS that cluster nodes run on.
`Podman`	Tool to build and run containers locally without a daemon.
`Istio`	A service mesh that manages, secures, and routes traffic between services.
`Helm`	A package manager for Kubernetes — bundles manifests into reusable charts.
`S2I (Source-to-Image)`	Red Hat tool that builds container images directly from source code.
`SCC (Security Context Constraints)`	OpenShift's security policies — controls what pods can do.
`OVN-Kubernetes`	The default network provider in OpenShift 4.10+ — manages pod networking.
`CNI`	Container Network Interface — standard for network plugins in Kubernetes.
`PV`	Persistent Volume — the actual storage resource, cluster-wide.
`PVC`	Persistent Volume Claim — a pod's request for storage from a PV.
`StorageClass`	Describes a type of storage — defines provisioner, reclaim policy, etc.
`HPA`	Horizontal Pod Autoscaler — auto-scales pods based on CPU/memory.
`Image Stream`	OpenShift resource that tracks container images and their versions/tags.
`Floating tag`	An image tag like 'latest' or '1' that can change — points to different images over time.
`Fixed tag`	An image tag like '1-222' that always points to the exact same image build.
`Image Trigger`	Makes a deployment redeploy automatically when an image stream tag changes.
`Readiness Probe`	Checks if app is ready for traffic. Pod won't get traffic until probe passes.
`Liveness Probe`	Checks if app is still alive. Pod is restarted if probe fails.
`DaemonSet`	Runs one copy of a pod on every node in the cluster.

`StatefulSet`	Like a deployment but each pod gets its own unique storage and stable name.
`Job`	Runs a pod once to completion — not restarted after success.
`CronJob`	Runs a Job on a schedule (like cron on Linux).
`Route`	OpenShift-specific — exposes a service to the internet via a hostname.
`Ingress`	Kubernetes-standard — similar to Route, exposes services externally.
`ClusterIP`	Default service type — only accessible within the cluster.
`NodePort`	Exposes service on a port on every node (30000-32767 range).
`LoadBalancer`	Provisions a cloud load balancer to expose the service externally.
`Headless Service`	Service with clusterIP: None — used by StatefulSets for direct pod DNS.
`ConfigMap`	Stores non-sensitive configuration data (files, key-value pairs) for pods.
`Secret`	Stores sensitive data (passwords, tokens) — Base64 encoded, access restricted.
`Replica`	A copy of a pod. Multiple replicas = high availability + load balancing.
`RWO`	ReadWriteOnce — one node mounts as read/write.
`RWX`	ReadWriteMany — many nodes can read and write (NFS).
`RWOP`	ReadWriteOncePod — only one pod in entire cluster can mount as read/write.

End of DO180 Command Reference Guide